

Python for Data Science

Buổi 5: Pandas Nâng Cao I – Missing Data & Kết Hợp Dữ Liệu

PGS-TS Nguyễn Mạnh Hùng

August 19, 2026

Cấu trúc Buổi 5

- 1 Ôn tập Buổi 3 và đặt vấn đề
- 2 Module 17: Xử lý dữ liệu thiếu (Missing Data)
- 3 Module 18: Hierarchical Indexing (MultiIndex)
- 4 Module 19: Kết hợp dữ liệu – Concat
- 5 Module 20: Kết hợp dữ liệu – Merge và Join
- 6 Bài tập áp dụng: hoàn thiện case study State Population & Baby Names

Lưu ý phạm vi

Buổi học tập trung xử lý dữ liệu thiếu và kết hợp nhiều bảng. Data cleaning nâng cao (duplicate, outlier), groupby và pivot table sẽ học ở Buổi 6.

Nhắc lại Buổi 3–4

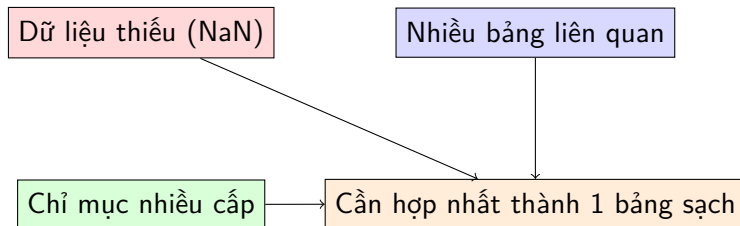
- Buổi 3 – Series/DataFrame: ndarray gắn nhãn (index, columns); `.loc[]/.iloc[]`, boolean indexing, `sort_values()`.
- Buổi 4 – đọc dữ liệu từ nhiều định dạng: CSV nâng cao, JSON, HTML, XML, Excel, Pickle/Parquet, Web API, SQL.
- Bài tập với *US Baby Names* (nhiều file theo năm) và *US State Population* (dữ liệu cần kết hợp với bảng tên bang).

Hai vấn đề còn bỏ ngỏ từ Buổi 3

- ❶ Làm sao ghép nhiều file `yobYYYY.txt` thành một DataFrame duy nhất?
- ❷ Làm sao "nối" `state-population.csv` với `state-abbrevs.csv` để có tên bang đầy đủ?

Buổi 5 trả lời trực tiếp cả hai câu hỏi này.

Dữ liệu thực tế: thiếu và phân mảnh



Ba chủ đề của Buổi 5 – xử lý thiếu dữ liệu, chỉ mục nhiều cấp, và kết hợp bảng – đều nhằm một mục tiêu chung: đưa dữ liệu thô, phân mảnh về một DataFrame sạch, sẵn sàng phân tích.

Module 17: Xử lý dữ liệu thiếu (Missing Data)

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Giải thích cách Pandas biểu diễn dữ liệu thiếu (NaN, None).
- 2 Phát hiện dữ liệu thiếu bằng `isnull()/notnull()`.
- 3 Loại bỏ hoặc điền dữ liệu thiếu bằng `dropna()` và `fillna()`.

NaN và None: hai kiểu "thiếu dữ liệu"

```
import numpy as np
import pandas as pd

s = pd.Series([1, np.nan, 3, None])
print(s)
# 0    1.0
# 1    NaN
# 2    3.0
# 3    NaN
# dtype: float64
```

- None: đối tượng Python, khi đưa vào Series số sẽ được Pandas tự động chuyển thành NaN.
- NaN (Not a Number): giá trị đặc biệt của float64 (chuẩn IEEE 754) – lý do cột chứa NaN thường bị ép kiểu float, ngay cả khi dữ liệu gốc là số nguyên.

Vì sao Pandas không dùng None cho dữ liệu số?

```
import numpy as np

vals1 = np.array([1, None, 2, 3])
print(vals1.dtype)          # object -- mọi phần tử là 1 đối tượng Python

vals1.sum()
# TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

Kết quả kiểm chứng (notebook 03.04-Missing-Values)

Mảng chứa None bị ép thành dtype=object (mảng đối tượng Python thuần) – các phép toán không còn được NumPy tăng tốc, và phép tổng trực tiếp gây lỗi TypeError. Đây là lý do Pandas chọn NaN (một giá trị float hợp lệ theo IEEE 754) làm sentinel chính cho dữ liệu số, thay vì None.

Phát hiện dữ liệu thiếu

```
df = pd.DataFrame({
    "a": [1, np.nan, 3],
    "b": [np.nan, 2, 3],
    "c": [1, 2, np.nan]
})

print(df.isnull())          # bang boolean: True tại vị trí thiếu
print(df.notnull())         # ngược lại với isnull()

print(df.isnull().sum())    # đếm số giá trị thiếu theo từng cột
print(df.isnull().sum().sum()) # tổng số giá trị thiếu toàn bảng
```

`isnull()` kết hợp `sum()` là bước kiểm tra đầu tiên bắt buộc trên bất kỳ tập dữ liệu thực tế nào trước khi phân tích.

Loại bỏ dữ liệu thiếu: dropna()

```
print(df.dropna())           # bỏ hàng có bất kỳ NaN nào
print(df.dropna(axis=1))     # bỏ cột có bất kỳ NaN nào
print(df.dropna(how="all"))  # chỉ bỏ hàng toàn NaN
print(df.dropna(thresh=2))   # giữ hàng có ít nhất 2 giá trị hợp lệ
```

Cần nhắc

`dropna()` mặc định rất "ng nghiêm khắc" (bỏ cả hàng chỉ thiếu 1 giá trị) – với dữ liệu thực tế nhiều cột, cách này có thể xóa mất phần lớn dữ liệu. Cần nhắc `thresh` hoặc `fillna()` thay thế.

Điền dữ liệu thiếu: fillna()

```
print(df.fillna(0))           # dien gia tri co dinh
print(df.fillna(df.mean()))   # dien bang trung binh tung cot

print(df.ffill())             # forward fill: lay gia tri phia truoc
print(df.bfill())             # backward fill: lay gia tri phia sau

df["a"] = df["a"].fillna(df["a"].median()) # dien rieng cho 1 cot
```

Lựa chọn chiến lược điền dữ liệu (0, trung bình, forward-fill...) phụ thuộc vào *bản chất* của dữ liệu – ví dụ dữ liệu chuỗi thời gian thường phù hợp với `ffill()` hơn là điền 0.

Giải pháp hiện đại: kiểu nullable Int32 và pd.NA

```
s = pd.Series([1, np.nan, 2, None, pd.NA], dtype="Int32")
print(s)
# 0      1
# 1    <NA>
# 2      2
# 3    <NA>
# 4    <NA>
# dtype: Int32
```

Nhược điểm của NaN là ép cột số nguyên thành float64 (đã thấy ở phần "NaN và None"). Các kiểu *nullable* viết hoa chữ cái đầu (Int32, Int64, Float64...) cùng sentinel pd.NA giữ nguyên ngữ nghĩa "số nguyên có thể thiếu" – tương thích với toàn bộ isnull(), dropna(), fillna() đã học. Đây là lựa chọn được khuyến nghị cho dữ liệu mới (yêu cầu khai báo dtype tường minh).

Tổng kết Module 17

- NaN là giá trị float đặc biệt biểu diễn dữ liệu thiếu; None tự động được chuyển thành NaN trong cột số (mảng chứa None thuần bị ép dtype=object, gây lỗi khi tính tổng).
- `isnull()/notnull()`: phát hiện; luôn kiểm tra bằng `.sum()` trước khi xử lý.
- `dropna()`: loại bỏ (chú ý axis, how, thresh).
- `fillna()`, `ffill()`, `bfill()`: điền giá trị thay thế.
- Kiểu nullable (Int32...) + `pd.NA`: tránh ép kiểu về float64 khi dữ liệu gốc là số nguyên.

Module 18: Hierarchical Indexing (MultiIndex)

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Giải thích vì sao cần chỉ mục nhiều cấp (MultiIndex).
- 2 Tạo và truy cập dữ liệu qua MultiIndex.
- 3 Chuyển đổi qua lại giữa dạng "dài" (long) và "rộng" (wide) bằng `stack()/unstack()`.

Vấn đề: dùng tuple Python làm index

```
index = [("California", 2010), ("California", 2020),  
         ("New York", 2010), ("New York", 2020),  
         ("Texas", 2010), ("Texas", 2020)]  
populations = [37253956, 39538223, 19378102, 20201249, 25145561, 29145505]  
pop = pd.Series(populations, index=index)  
  
# Lay tat ca gia tri nam 2010: phai loc thu cong, cham va "xau"  
pop[[i for i in pop.index if i[1] == 2010]]
```

Cách này *chạy được* nhưng không tận dụng cú pháp `.loc[]` / slicing quen thuộc của Pandas, và kém hiệu quả với dữ liệu lớn (nguồn: notebook 03.05-Hierarchical-Indexing, dân số 3 bang theo Tổng điều tra Dân số Hoa Kỳ 2010 & 2020).

Vì sao cần chỉ mục nhiều cấp?

Dữ liệu dân số có *hai* chiều phân loại tự nhiên: bang và năm. Một Index một chiều không biểu diễn đủ.

Giải pháp

MultiIndex cho phép một trục (hàng hoặc cột) mang **nhiều cấp nhãn** cùng lúc – ví dụ (state, year) – mà vẫn giữ dữ liệu trong một Series/DataFrame một cách gọn gàng, với cú pháp `.loc[]`/slicing như index một cấp.

Đây chính xác là vấn đề gặp phải với `state-population.csv` ở Buổi 3: mỗi bang có nhiều dòng theo year và ages.

Tạo MultiIndex

```
index = pd.MultiIndex.from_tuples(
    [("CA", 2000), ("CA", 2010), ("NY", 2000), ("NY", 2010)],
    names=["state", "year"]
)
pop = pd.Series(
    [33871648, 37253956, 18976457, 19378102],
    index=index
)
print(pop)
```

#	state	year	
#	CA	2000	33871648
#		2010	37253956
#	NY	2000	18976457
#		2010	19378102

Ngoài `from_tuples`, còn có thể tạo MultiIndex bằng `set_index(["state", "year"])` trực tiếp từ DataFrame có sẵn nhiều cột.

Truy cập dữ liệu qua MultiIndex

```
print(pop["CA"])           # tat ca cac nam cua bang CA -> Series con
print(pop["CA", 2000])     # 1 gia tri cu the: 33871648
print(pop.loc["CA":"NY"])  # slicing theo cap ngoai cung (index da sap xep)

print(pop.loc[:, 2010])    # partial indexing: tat ca bang, nam 2010
```

Truy cập theo cấp ngoài (state) trả về Series con giữ nguyên cấp còn lại (year) – tính chất "đệ quy" đặc trưng của MultiIndex.

stack() và unstack(): chuyển đổi Long $\leftrightarrow$ Wide

```
df_wide = pop.unstack()          # MultiIndex Series -> DataFrame 2 chiều
print(df_wide)
#           2000           2010
# CA  33871648  37253956
# NY  18976457  19378102

df_long = df_wide.stack()         # DataFrame -> lại thành Series có MultiIndex
```

Ứng dụng

`unstack()` rất hữu ích để *trình bày* dữ liệu (bảng bang $\times$ năm dễ đọc); dạng "long" (một dòng = một quan sát) lại thường thuận tiện hơn cho `groupby` và trực quan hóa (sẽ học ở Buổi 6, 8–9).

MultiIndex trên cả hai trục: dữ liệu y tế

```
index = pd.MultiIndex.from_product([[2013, 2014], [1, 2]], names=["year", "visit"])
columns = pd.MultiIndex.from_product(
    [["Bob", "Guido", "Sue"], ["HR", "Temp"]], names=["subject", "type"])
health_data = pd.DataFrame(data, index=index, columns=columns)
print(health_data)
# subject      Bob      Guido      Sue
# type         HR  Temp   HR  Temp   HR  Temp
# year visit
# 2013 1       40.0  36.0  45.0  37.9  17.0  35.7
#       2       38.0  36.7  37.0  36.1  46.0  37.8
# 2014 1       38.0  38.1  42.0  36.1  41.0  36.0
#       2       46.0  37.0  35.0  36.3  49.0  36.8

print(health_data["Guido"])  # lay toan bo du lieu (HR, Temp) cua 1 nguoi
```

Dữ liệu 4 chiều (bệnh nhân $\times$ chỉ số $\times$ năm $\times$ lần khám) được biểu diễn gọn trong 1 bảng 2 chiều nhờ MultiIndex ở *cả hàng lẫn cột* (nguồn: notebook 03.05-Hierarchical-Indexing, mục "MultiIndex for columns").

Chuyển đổi Index $\leftrightarrow$ cột: `reset_index()` và `set_index()`

```
pop_flat = pop.reset_index(name="population")    # index -> cot thuong
print(pop_flat)
#           state  year  population
# 0  California  2010    37253956
# 1  California  2020    39538223
# 2    New York  2010    19378102
# ...

pop_flat.set_index(["state", "year"])            # cot -> lai thanh MultiIndex
```

Cặp thao tác `reset_index()/set_index()` rất thường gặp khi khám phá dữ liệu thực tế: đưa index về dạng cột thường để dễ lọc/merge, rồi đặt lại chỉ mục nhiều cấp khi cần trình bày hoặc slicing.

Tổng kết Module 18

- `MultiIndex`: chỉ mục nhiều cấp, biểu diễn dữ liệu nhiều chiều phân loại trong 1 `Series/DataFrame`.
- Tạo bằng `MultiIndex.from_tuples()` hoặc `set_index(...)`.
- Truy cập theo cấp ngoài trước, cấp trong sau; `.loc[]` hỗ trợ tuple và slicing.
- `unstack()`: long $\rightarrow$ wide; `stack()`: wide $\rightarrow$ long.
- Cả hàng lẫn cột đều có thể mang `MultiIndex` cùng lúc (dữ liệu nhiều chiều trong 1 bảng 2D).
- `reset_index()/set_index()`: chuyển đổi qua lại giữa index và cột thường.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Nối nhiều DataFrame theo hàng hoặc theo cột bằng `pd.concat()`.
- 2 Xử lý tình huống index bị trùng lặp sau khi nối.
- 3 Áp dụng `concat()` để gộp nhiều file dữ liệu cùng cấu trúc.

pd.concat(): nối theo hàng

```
df_a = pd.DataFrame({"name": ["An", "Binh"], "age": [25, 32]})
df_b = pd.DataFrame({"name": ["Chi", "Dung"], "age": [41, 29]})

print(pd.concat([df_a, df_b]))
#   name  age
# 0   An   25
# 1  Binh   32
# 0   Chi   41    <- index bị lặp lại!
# 1  Dung   29

print(pd.concat([df_a, df_b], ignore_index=True))    # danh lại index từ 0
```

Lưu ý

Từ các phiên bản Pandas gần đây, phương thức `DataFrame.append()` đã bị loại bỏ hoàn toàn – `pd.concat([...])` là cách chuẩn duy nhất để nối `DataFrame`.

Bắt lỗi index trùng lặp: `verify_integrity`

```
x = pd.DataFrame({"A": ["A0", "A1"], "B": ["B0", "B1"]})
y = pd.DataFrame({"A": ["A2", "A3"], "B": ["B2", "B3"]})
y.index = x.index          # có tình trạng index trùng nhau

pd.concat([x, y], verify_integrity=True)
# ValueError: Indexes have overlapping values: Index([0, 1], dtype='int64')
```

Mặc định `pd.concat()` *giữ nguyên* index kể cả khi trùng lặp (khác với `np.concatenate`) – đây thường là điều không mong muốn. `verify_integrity=True` biến việc trùng index thành lỗi tường minh thay vì một bug âm thầm; `ignore_index=True` (trang sau) là cách khắc phục phổ biến nhất.

concat(): giữ nguồn gốc dữ liệu với keys

```
combined = pd.concat([df_a, df_b], keys=["batch1", "batch2"])
print(combined)
#           name  age
# batch1 0      An   25
#         1     Binh  32
# batch2 0      Chi  41
#         1     Dung  29

print(combined.loc["batch1"])    # lay lai rieng batch1
```

Tham số `keys` tạo thêm một cấp MultilIndex, hữu ích khi cần *truy vết* dữ liệu về nguồn gốc (ví dụ: file/năm nào) sau khi đã gộp chung.

concat() theo cột (axis=1)

```
df_score = pd.DataFrame({"score": [90, 85]})
print(pd.concat([df_a, df_score], axis=1))
#    name  age  score
# 0    An   25     90
# 1  Binh   32     85
```

axis=1 nối theo cột (yêu cầu cùng số hàng/khớp index) – tương tự np.hstack() đã học ở Buổi 2, nhưng có căn chỉnh theo index thay vì chỉ theo vị trí.

concat() khi cột không khớp hoàn toàn: tham số join

```
df5 = pd.DataFrame({"A": ["A1","A2"], "B": ["B1","B2"], "C": ["C1","C2"]})
df6 = pd.DataFrame({"B": ["B3","B4"], "C": ["C3","C4"], "D": ["D3","D4"]})

print(pd.concat([df5, df6]))           # join="outer" (mặc định): hợp cột, điền NaN
#    A   B   C   D
# 1  A1  B1  C1  NaN
# 2  A2  B2  C2  NaN
# 3  NaN B3  C3  D3
# 4  NaN B4  C4  D4

print(pd.concat([df5, df6], join="inner")) # chỉ giữ cột chung (B, C)
```

Khi hai nguồn dữ liệu có *tập cột khác nhau* (thường gặp khi gộp file từ nhiều năm/nguồn), `join="outer"` (mặc định) giữ mọi cột và điền NaN; `join="inner"` chỉ giữ phần giao nhau – lựa chọn cần cân nhắc kỹ trước khi gộp dữ liệu thực tế.

Ứng dụng: gộp nhiều file US Baby Names

```
years = [1980, 1990, 2000, 2010]
frames = []
for year in years:
    df_year = pd.read_csv(
        f"datasets/babynames/yob{year}.txt",
        names=["name", "sex", "births"]
    )
    df_year["year"] = year          # danh dau nam truoc khi gop
    frames.append(df_year)

names_all = pd.concat(frames, ignore_index=True)
print(names_all.shape)
print(names_all.groupby("year")["births"].sum())    # preview groupby - hoc ky o Buoi 5
```

Đây chính là câu trả lời cho phần "Mở rộng" của Bài tập 2, Buổi 3: thêm cột year trước khi concat để không mất thông tin nguồn gốc từng dòng.

Tổng kết Module 19

- `pd.concat(...)`: cách chuẩn để nối nhiều DataFrame (thay cho `.append()` đã bị loại bỏ).
- Mặc định giữ nguyên index kể cả khi trùng lặp; `verify_integrity=True` biến trùng lặp thành lỗi tường minh.
- `ignore_index=True`: đánh lại index từ đầu; `keys=...`: giữ nguồn gốc dữ liệu.
- `axis=1`: nối theo cột thay vì theo hàng; `join="outer"/"inner"`: cách xử lý cột không khớp hoàn toàn.
- Kỹ thuật chuẩn để gộp nhiều file cùng cấu trúc (nhiều năm, nhiều batch).

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Phân biệt `concat()` (nối theo trục) và `merge()` (kết hợp theo khóa/key, giống SQL JOIN).
- 2 Sử dụng `pd.merge()` với các kiểu `how`: `inner`, `outer`, `left`, `right`.
- 3 Áp dụng `merge()` để kết hợp nhiều bảng dữ liệu thực tế.

concat() vs merge(): khác nhau ở đâu?

	<code>pd.concat()</code>	<code>pd.merge()</code>
Nguyên lý	Xếp chồng theo trục	Khớp theo giá trị khóa (key)
Giống	<code>np.vstack/hstack</code>	SQL JOIN
Yêu cầu	Cùng cấu trúc cột/hàng	Có cột/khóa chung để khớp

`state-population.csv` và `state-abbrevs.csv` không thể "xếp chồng" bằng `concat` – chúng cần được *khớp* với nhau qua cột mã bang, đây chính là việc của `merge()`.

Ba loại merge: one-to-one, many-to-one, many-to-many

```
df1 = pd.DataFrame({"employee": ["Bob","Jake","Lisa","Sue"],
                    "group": ["Accounting","Engineering","Engineering","HR"]})
df2 = pd.DataFrame({"employee": ["Lisa","Bob","Jake","Sue"],
                    "hire_date": [2004, 2008, 2012, 2014]})
df3 = pd.merge(df1, df2)          # one-to-one: khoa "employee" không lặp ở cả 2 bên

df4 = pd.DataFrame({"group": ["Accounting","Engineering","HR"],
                    "supervisor": ["Carly","Guido","Steve"]})
pd.merge(df3, df4)               # many-to-one: "group" lặp ở df3, không lặp ở df4

df5 = pd.DataFrame({"group": ["Accounting","Accounting","Engineering",
                              "Engineering","HR","HR"],
                    "skills": ["math","spreadsheets","software","math",
                              "spreadsheets","organization"]})
pd.merge(df1, df5)               # many-to-many: "group" lặp ở cả 2 bên
```

`pd.merge()` tự động nhận diện loại join dựa trên *dữ liệu* của khóa, không cần khai báo tường minh – nguồn gốc từ đại số quan hệ (relational algebra) dùng trong cơ sở dữ liệu (notebook 03.07-Merge-and-Join).

pd.merge(): kết hợp cơ bản

```
pop = pd.DataFrame({
    "state/region": ["CA", "CA", "NY"],
    "year": [2000, 2010, 2010],
    "population": [33871648, 37253956, 19378102]
})
abbrevs = pd.DataFrame({
    "state": ["California", "New York"],
    "abbreviation": ["CA", "NY"]
})

merged = pd.merge(pop, abbrevs, how="left",
                  left_on="state/region", right_on="abbreviation")
print(merged.drop(columns=["abbreviation"]))
#  state/region  year  population      state
#  0           CA  2000    33871648  California
#  1           CA  2010    37253956  California
#  2           NY  2010    19378102    New York
```

left_on/right_on dùng khi tên cột khóa khác nhau giữa hai bảng (state/region vs abbreviation).

Bốn kiểu merge: how=

how=	Kết quả	Tương tự SQL
"inner" (mặc định)	Chỉ giữ khóa có ở cả hai bảng	INNER JOIN
"outer"	Giữ tất cả khóa, điền NaN nếu thiếu	FULL OUTER JOIN
"left"	Giữ toàn bộ bảng trái, khớp gì có vậy	LEFT JOIN
"right"	Giữ toàn bộ bảng phải	RIGHT JOIN

Lưu ý quan trọng

how="inner" (mặc định) có thể **âm thầm làm mất dữ liệu** nếu khóa không khớp hoàn toàn giữa hai bảng. Luôn kiểm tra shape trước/sau khi merge, và cân nhắc how="left" hoặc "outer" khi không chắc chắn.

Cột trùng tên ở cả hai bảng: tham số suffixes

```
df8 = pd.DataFrame({"name": ["Bob","Jake","Lisa","Sue"], "rank": [1, 2, 3, 4]})
df9 = pd.DataFrame({"name": ["Bob","Jake","Lisa","Sue"], "rank": [3, 1, 4, 2]})

print(pd.merge(df8, df9, on="name"))
#   name rank_x rank_y      <- Pandas tự thêm hậu tố _x/_y
# 0  Bob      1      3
# ...

print(pd.merge(df8, df9, on="name", suffixes=["_L", "_R"]))  # hậu tố tùy chỉnh
```

Khi hai bảng có cột trùng tên ngoài khóa (ở đây là rank), merge() tự thêm hậu tố _x/_y; tham số suffixes cho phép đặt hậu tố có ý nghĩa hơn (ví dụ _L/_R hoặc _2023/_2024).

Merge nhiều bảng liên tiếp

```
areas = pd.DataFrame({
    "state": ["California", "New York"],
    "area (sq. mi)": [423967, 54555]
})

final = pd.merge(merged, areas, on="state", how="left")
final["density"] = final["population"] / final["area (sq. mi)"]
print(final[["state", "year", "population", "density"]])
```

Kết hợp **3 bảng** (state-population, state-abbrevs, state-areas) qua hai lượt merge() liên tiếp – đây chính xác là quy trình để tính *mật độ dân số* từ dữ liệu thô của Buổi 3.

join(): merge theo index

```
left = pd.DataFrame({"x": [1, 2, 3]}, index=["a", "b", "c"])
right = pd.DataFrame({"y": [4, 5, 6]}, index=["a", "b", "d"])

print(left.join(right, how="outer"))
#      x      y
# a  1.0  4.0
# b  2.0  5.0
# c  3.0  NaN
# d  NaN  6.0
```

.join() là phiên bản rút gọn của merge() khi khóa kết hợp chính là **index** của cả hai bảng, thường gặp sau khi dùng set_index() hoặc groupby().

- `merge()`: kết hợp theo khóa (giống SQL JOIN), khác `concat()` (xếp chồng theo trục).
- Ba loại join tùy theo khóa có trùng lặp hay không: one-to-one, many-to-one, many-to-many – Pandas tự nhận diện từ dữ liệu.
- `on`, `left_on`/`right_on`: chỉ định cột khóa; `suffixes`: đặt hậu tố cho cột trùng tên.
- `how`: `inner` (mặc định), `outer`, `left`, `right` – luôn kiểm tra shape sau khi merge.
- `.join()`: merge theo index, cú pháp gọn hơn.

Ví dụ đầy đủ: xử lý null sau merge (state-population)

```
pop = pd.read_csv("data/state-population.csv")
areas = pd.read_csv("data/state-areas.csv")
abbrevs = pd.read_csv("data/state-abbrevs.csv")

merged = pd.merge(pop, abbrevs, how="outer",
                  left_on="state/region", right_on="abbreviation")
merged = merged.drop("abbreviation", axis=1)

merged.loc[merged["state"].isnull(), "state/region"].unique()
# array(['PR', 'USA'], dtype=object)    <- 2 mã không có trong bảng abbrevs

merged.loc[merged["state/region"] == "PR", "state"] = "Puerto Rico"
merged.loc[merged["state/region"] == "USA", "state"] = "United States"
```

`how="outer"` giữ mọi bảng để không âm thầm mất dữ liệu; sau đó dùng `isnull()` để tìm và *sửa tận gốc* 2 mã vùng đặc biệt (PR, USA) không khớp được với bảng `abbrevs` – kỹ thuật gỡ lỗi merge kinh điển (nguồn: notebook 03.07-Merge-and-Join, mục "Example: US States Data", đã kiểm chứng lại trên dữ liệu thật).

Ví dụ đầy đủ: xếp hạng mật độ dân số 50 bang

```
final = pd.merge(merged, areas, on="state", how="left")
final.dropna(inplace=True)          # bỏ "United States" (không có diện tích riêng)

data2010 = final.query("year == 2010 & ages == 'total'").set_index("state")
density = (data2010["population"] / data2010["area (sq. mi)"]).sort_values(
    ascending=False)

print(density.head())               # 5 khu vực mật độ dân số cao nhất
print(density.tail())               # 5 bang mật độ dân số thấp nhất
```

Kết quả kiểm chứng thực tế: đông đúc nhất là *District of Columbia* ($\approx 8\,899$ người/dặm²), tiếp theo Puerto Rico, và trong số các bang, *New Jersey* đông nhất ($\approx 1\,009$ người/dặm²); thưa dân nhất là *Alaska* ($\approx 1,09$ người/dặm²).

Bài tập 1: Mở rộng case study State Population

Dữ liệu (đã dùng ở Buổi 3, kho *PythonDataScienceHandbook*, notebooks/data/):
`state-population.csv`, `state-abbrevs.csv`, `state-areas.csv`. Tự chạy lại pipeline
merge ở 2 trang trước, sau đó mở rộng:

- 1 Lặp lại quy trình với `ages == "under18"` thay vì `"total"` – thứ hạng mật độ trẻ em dưới 18 tuổi có khác so với dân số tổng không?
- 2 So sánh mật độ dân số bang *California* qua các năm có trong dữ liệu (dùng `unstack()` trên kết quả `set_index(["state", "year"])`) – xu hướng tăng/giảm ra sao?
- 3 Thử đổi `how="outer"` thành `how="left"` ở bước merge với `state-abbrevs.csv` – kết quả `isnull().sum()` có gì khác? Vì sao?
- 4 Dùng `density.rank(ascending=False)` để lấy thứ hạng số của từng bang; tự tra cứu mật độ dân số TP.HCM và so sánh để có góc nhìn thực tế về thang đo "người/dặm²".

Bài tập 2: Gộp dữ liệu US Baby Names nhiều năm

Dữ liệu (kho *python-for-data-analysis*, *datasets/babynames/*).

- 1 Đọc 3–4 file `yobYYYY.txt` khác nhau (ví dụ 1980, 1990, 2000, 2010), thêm cột `year` cho mỗi file.
- 2 Dùng `pd.concat()` với `ignore_index=True` để gộp thành một `DataFrame` duy nhất.
- 3 Dùng `unstack()/pivot` (gợi ý: `set_index(["name", "year"])["births"].unstack()`) để xem một cái tên cụ thể (ví dụ "Emma") thay đổi độ phổ biến qua các năm như thế nào.
- 4 Kiểm tra `isnull()` sau `unstack()`: giá trị `NaN` ở đây có ý nghĩa gì? Nên `fillna(0)` hay giữ nguyên `NaN`?

Câu hỏi thảo luận

Việc `fillna(0)` cho số lượt đặt tên có hợp lý không? So sánh với việc `fillna(0)` cho một cột điểm số trung bình – vì sao chiến lược điền dữ liệu cần phụ thuộc vào ý nghĩa của cột?

Bảng tổng hợp các thao tác Pandas trong Buổi 5

Nhóm	Hàm/thao tác	Mục đích
Missing data	<code>isnull</code> , <code>notnull</code> , <code>dropna</code> , <code>fillna</code>	Phát hiện, loại bỏ, điền dữ liệu thiếu
Missing data	Kiểu nullable (<code>Int32...</code>) + <code>pd.NA</code>	Giữ kiểu số nguyên khi có giá trị thiếu
MultiIndex	<code>MultiIndex.from_tuples</code> , <code>set_index</code>	Chỉ mục nhiều cấp (hàng và/hoặc cột)
Long/Wide	<code>stack</code> , <code>unstack</code> , <code>reset_index</code>	Chuyển đổi hình dạng dữ liệu
Kết hợp (trục)	<code>pd.concat</code> (<code>join</code> , <code>verify_integrity</code>)	Xếp chồng theo hàng/cột
Kết hợp (khóa)	<code>pd.merge</code> (<code>how</code> , <code>suffixes</code>), <code>.join</code>	Kết hợp theo khóa, giống SQL JOIN

Tài liệu tham khảo I

- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56.
- VanderPlas, J. (n.d.). *PythonDataScienceHandbook* [Kho mã nguồn – dữ liệu state-population.csv, state-abbrevs.csv, state-areas.csv]. GitHub.
<https://github.com/jakevdp/PythonDataScienceHandbook>
- Pugh, D. R. (n.d.). *python-for-data-analysis* [Kho mã nguồn – dữ liệu babynames]. GitHub.
<https://github.com/davidrpugh/python-for-data-analysis>
- The pandas development team. (2020). *pandas-dev/pandas: Pandas (Version 1.0)* [Software]. Zenodo.
<https://doi.org/10.5281/zenodo.3509134>

Chuẩn bị cho buổi tiếp theo: Pandas Nâng Cao II – Data Cleaning, GroupBy & Pivot Table

Thông điệp

Missing data, MultiIndex, concat và merge là bốn kỹ năng nền tảng để biến dữ liệu thô, phân mảnh từ nhiều nguồn thành một DataFrame sạch, sẵn sàng cho bước tổng hợp (groupby) và trực quan hóa.